

# Twelve-Month Curriculum for Top-Paying AI Infrastructure Engineering Roles

## Executive summary

Your highest-probability path to a **stable** engineering role paying roughly **US\$700k–1M+ total compensation** is not generic “study everything” interview prep. It is a **staff-caliber AI infrastructure curriculum** aimed at the role families that the market is currently paying for: **LLM inference and serving, ML platform and evaluation systems, distributed backend infrastructure, data infrastructure, and reliability/performance engineering for AI products**. Recent U.S. compensation data points show that this target range is already plausible at the upper individual-contributor bands of several firms: OpenAI SWE median compensation is about **\$800k** with higher levels above **\$1.2M**; Anthropic’s senior/lead SWE bands are around **\$563k–\$841k**; Databricks L6 staff engineering is around **\$1.05M**; Meta E6/E7 software engineering is about **\$703k/\$1.24M**; Google L7/L8 is about **\$829k/\$1.12M**; Stripe L4/L5 is about **\$760k/\$929k**; Snowflake IC5 reaches roughly **\$1.18M**; and NVIDIA IC7 is around **\$1.07M**. These figures are directional, not guarantees, because they are crowd-sourced and especially sensitive to private-company equity valuation. <sup>1</sup>

Current openings at those companies also point to the same conclusion: the premium is concentrated in roles that combine **distributed systems, production ML, latency/cost optimization, data infrastructure, evaluation infrastructure, and technical leadership**. OpenAI is hiring around data infrastructure, real-time systems, and compute infrastructure; Anthropic is actively hiring in inference, caching, databases, cloud inference, AI reliability, and research tooling; Google staff AI/ML roles explicitly call for full ML lifecycle ownership, genAI production systems, RAG, globally distributed databases, and inference cost/latency optimization; Databricks staff roles emphasize large-scale distributed systems, backend platform work, search quality, and AI platform engineering; Snowflake is hiring for Cortex AI infrastructure and feature-store/platform roles; Stripe’s ML-platform roles emphasize latency, reliability, cost, and security constraints; and NVIDIA is hiring for GPU-cluster and AI-infrastructure software roles. <sup>2</sup>

That market signal is why this curriculum prioritizes four things above everything else: **coding fluency, distributed systems and system design, AI systems and LLM-serving depth, and a behavioral portfolio that proves scope, judgment, and leadership**. Even the companies that lean more practical than algorithmic still use coding as a gate: Anthropic publicly says technical candidates should expect live coding in Colab or CodeSignal; Meta’s software and ML loops still start with technical screens and multi-round interviews; Microsoft says technical interviews evaluate problem solving, design, coding, and testing; and Databricks says it still uses algorithms and CS fundamentals while increasingly emphasizing hands-on coding, debugging, and design. <sup>3</sup>

My recommendation is to treat **10 hours per week** as the default operating mode, **5 hours** as maintenance mode, and **15 hours** as an aggressive fast-track. If you execute the full 12 months well, the target outcome should be: **120–160 high-yield coding problems, 20–30 system-design reps, 3 portfolio-grade mini-projects in AI infra/serving/evals, 20+ mocks, and a fully prepared 30-story behavioral bank**. That

combination is what best matches your current background as an experienced Microsoft Applied Scientist working on LLMs, telemetry, and distributed services.

## Market target and role fit

For your background, the best-fit route is **engineering-heavy Applied AI infrastructure**, not a pure research-scientist track. That is partly because the highest-paying “stable salaried” roles are currently clustered in senior IC engineering bands, and partly because current job descriptions reward exactly the kinds of work you already do: **operating production systems, instrumenting them, evaluating model quality, building internal platforms, and making ambiguous systems reliable at scale**. Google’s current staff ML job pages ask for large-scale ML infrastructure, full ML lifecycle ownership, generative AI in production, RAG, and inference cost/latency optimization. Databricks staff roles ask for architecting and operating large-scale distributed systems. Stripe staff ML-platform roles ask for arbitration under latency, reliability, cost, and security constraints. NVIDIA’s principal AI infrastructure roles center on GPU clusters and large-scale infrastructure. <sup>4</sup>

In practice, that means your “north-star” titles should be things like **Member of Technical Staff, Senior Staff Software Engineer, Staff/Principal Software Engineer, Staff Software Engineer, AI/ML, Systems ML / AI SysML, ML Platform Engineer, Inference Runtime Engineer, Cloud Inference Engineer, AI Reliability Engineer**, or **Data Infrastructure Engineer**. OpenAI explicitly notes that research and engineering hires share the internal title **Member of Technical Staff**. Anthropic’s public job board currently includes Staff+ software roles in platform, inference runtime, caching, databases, cloud inference, and reliability. Meta is hiring Systems ML and AI systems software engineering roles. Databricks, Snowflake, Stripe, and NVIDIA all publish senior/staff/principal openings for distributed data systems, AI platform, ML platform, or AI infrastructure work. <sup>5</sup>

The implication for your study plan is straightforward:

| Strategic conclusion                                                               | Why it matters                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optimize for <b>staff-level engineering judgment</b> , not just interview survival | The target compensation bands sit primarily at staff/principal/lead levels at public companies and staff-equivalent bands at frontier labs. <sup>6</sup>                                                                      |
| Put <b>AI systems + distributed systems</b> at the center of the curriculum        | Current role descriptions repeatedly call for distributed systems, inference optimization, data infrastructure, and platform work. <sup>7</sup>                                                                               |
| Keep coding prep <b>sharp but bounded</b>                                          | Coding is still a gate at Anthropic, Meta, Microsoft, and Databricks, but Databricks explicitly says it also cares about design, debugging, and realistic hands-on problem solving. <sup>8</sup>                              |
| Build an <b>evidence portfolio</b> , not only private notes                        | The most relevant roles reward demonstrable engineering taste: eval harnesses, serving benchmarks, infrastructure docs, and measurable system improvements. This is an inference from current role requirements. <sup>9</sup> |

## Prioritized curriculum

The priority stack below is intentionally opinionated. **P0** topics are the ones that most directly move both **interview odds** and **comp band**. **P1** topics materially improve staff-level performance and role fit. **P2** topics are useful, but they should never crowd out P0 work.

| Priority | Topic                              | Default share of a 10h week | Why this sits here                                                                                                           |
|----------|------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| P0       | Coding interviews                  | 2.5h                        | Direct screening gate at Anthropic, Meta, Microsoft, Databricks. <sup>8</sup>                                                |
| P0       | Distributed systems                | 2.0h                        | Core requirement in current Google, Databricks, Snowflake, Stripe, and NVIDIA roles. <sup>10</sup>                           |
| P0       | AI systems and LLM infra           | 2.0h                        | Best match to frontier-lab and top AI-platform roles. <sup>11</sup>                                                          |
| P0       | System design                      | 1.0h                        | The interview format and target levels reward architecture and tradeoff quality. <sup>12</sup>                               |
| P0       | Behavioral portfolio               | 1.0h                        | Staff-level loops evaluate collaboration, judgment, influence, adaptability, and scope. <sup>13</sup>                        |
| P1       | Databases and storage              | 0.7h                        | Repeatedly shows up in ML/AI infra roles and is central to design quality. <sup>14</sup>                                     |
| P1       | Networking                         | 0.4h                        | Necessary for debugging, low-latency serving, and API/system design. <sup>15</sup>                                           |
| P1       | Security                           | 0.4h                        | Critical for enterprise AI, platform roles, and public-sector/abuse/safeguards work. <sup>16</sup>                           |
| P1       | ML systems, evals, experimentation | 0.7h                        | LLM platform quality increasingly depends on evals, fine-tuning, distillation, retrieval, and experimentation. <sup>17</sup> |
| P1       | CS fundamentals and runtime depth  | 0.3h                        | Valuable for low-level systems, performance work, concurrency, and debugging subtle failures. <sup>18</sup>                  |

### Coding interviews — P0.

Why it matters: current public hiring materials show that live or interactive coding remains a direct gate at Anthropic, Meta, Microsoft, and Databricks. Your goal is not maximum volume; it is **pattern fluency under time pressure**, plus clean communication and testability. <sup>8</sup>

- **Study:** arrays and hashing; two pointers; sliding window; stacks and queues; binary search; trees and BST invariants; heaps; graphs; DFS/BFS; topological sort; union-find; intervals; greedy; dynamic programming fundamentals; recursion/backtracking.

- **Read and use:** LeetCode **Top Interview 150**, **LeetCode 75**, and **Graph Theory Study Plan**; Steven Skiena's **The Algorithm Design Manual** Part I for technique selection and pattern recognition. <sup>19</sup>
- **Practice set:** 1 *Two Sum*; 56 *Merge Intervals*; 57 *Insert Interval*; 76 *Minimum Window Substring*; 121 *Best Time to Buy and Sell Stock*; 128 *Longest Consecutive Sequence*; 146 *LRU Cache*; 200 *Number of Islands*; 207 *Course Schedule*; 215 *Kth Largest Element in an Array*; 239 *Sliding Window Maximum*; 295 *Find Median from Data Stream*; 347 *Top K Frequent Elements*; 560 *Subarray Sum Equals K*; 981 *Time Based Key-Value Store*. Use a strict rule: write tests or sample cases before finalizing the solution. <sup>20</sup>
- **Mini-projects:** create a “patterns notebook” repo with one-page writeups per pattern; build a timed mock harness that logs solve-time and bug categories; write a benchmark note comparing two valid solutions for three medium problems to practice tradeoff explanation.

### CS fundamentals and runtime depth — P1.

Why it matters: Databricks explicitly says low-level systems roles emphasize multithreading and OS primitives; Google, NVIDIA, and similar infra roles still reward engineers who understand scheduling, memory, concurrency, and performance, not just APIs. <sup>21</sup>

- **Study:** processes vs threads; synchronization primitives; deadlocks; condition variables; memory hierarchy; virtual memory and page faults; file-system persistence; Python `asyncio` vs threads vs multiprocessing; one systems language beyond Python if needed for target roles, ideally Go or C++ depending on the company mix.
- **Read and use: Operating Systems: Three Easy Pieces** sections on virtualization, concurrency, and persistence; Berkeley **CS 162** materials; MIT **6.5840** background readings for implementation habits. <sup>22</sup>
- **Practice set:** implement a bounded blocking queue; build a thread pool; compare Python threading vs multiprocessing on CPU and I/O tasks; induce and debug a deadlock; profile a toy service with flame graphs; implement token-bucket and leaky-bucket rate limiters; write a consistent-hashing library; benchmark lock contention; add structured logging and benchmark overhead; write a short note explaining a production incident in terms of OS/runtime behavior.
- **Mini-projects:** ship a concurrency-lab repo with benchmarks; write a “Python runtime tradeoffs for service engineers” memo; build a tiny rate-limited async crawler with backpressure.

### Distributed systems — P0.

Why it matters: this is one of the strongest common denominators across Google Staff AI/ML, Databricks staff backend, Snowflake Cortex infrastructure, Stripe ML platform, and NVIDIA AI infrastructure roles. These jobs are effectively interviews in **replication, partitioning, durability, latency, and failure handling**. <sup>10</sup>

- **Study:** replication; quorum thinking; partitioning and rebalancing; consistency models; consensus; leader election; failure detectors; retries/timeouts/circuit breaking; idempotency; backpressure; stream processing; exactly-once myths; CAP as a tradeoff lens, not a slogan; SLOs and graceful degradation.
- **Read and use: Designing Data-Intensive Applications** Chapters 1–9 and 11; MIT **6.5840** lectures and labs; Ongaro and Ousterhout's **Raft** paper. <sup>23</sup>
- **Practice set:** MIT MapReduce lab; explain Raft election and safety from memory; design a shard rebalancer; design an idempotent job processor; compare primary-replica vs quorum systems; write a failure-mode matrix for a multi-region API; design a deduplicating event pipeline; choose timeouts and retry budgets for a fan-out service; explain read-your-writes vs monotonic reads with examples; critique an architecture for split-brain risk.

- **Mini-projects:** implement a tiny replicated log or simplified Raft toy; build an idempotent asynchronous worker queue with dedupe keys; write a production-style design doc for a multi-tenant job orchestration service.

## AI systems and LLM infrastructure — P0.

Why it matters: this is the category with the strongest overlap between your background and current premium-paying roles. OpenAI, Anthropic, Google, Databricks, Snowflake, Stripe, and NVIDIA all publish roles or technical materials centered on inference, serving, evals, retrieval, reliability, and production ML systems. <sup>24</sup>

- **Study:** transformer internals; tokenization; embeddings; vector retrieval; reranking; RAG failure modes; context windows and context management; KV cache; continuous batching; PagedAttention; FlashAttention; quantization; speculative decoding; distillation; model routing; tool use and agent loops; eval design; latency/cost budgeting; multi-tenant serving and quotas.
- **Read and use:** **Attention Is All You Need**; **FlashAttention**; **PagedAttention/vLLM**; **Fast Inference from Transformers via Speculative Decoding**; OpenAI guides on **evals**, **model optimization**, and **fine-tuning/distillation**; Anthropic on **Building Effective Agents**, **Contextual Retrieval**, **Effective Context Engineering**, and **Demystifying Evals**; Google Research and Google blog posts on speculative decoding and inference optimization. <sup>25</sup>
- **Practice set:** compute KV-cache sizing for several model/context tuples; benchmark vLLM throughput vs latency under batch-size changes; run a contextual-retrieval ablation with and without reranking; design a multi-tenant inference scheduler with quotas; build a speculative-decoding benchmark; compare quantized vs unquantized latency/cost; create an eval set for a RAG or agent system; run a tiny distillation experiment; design a model-fallback cascade; write an SLO dashboard for an inference API.
- **Mini-projects:** build a portfolio-grade **LLM serving benchmark harness**; build a **RAG + eval pipeline** with retrieval diagnostics; build a **latency/cost optimization report** comparing batching, caching, and quantization choices.

## Databases and storage — P1.

Why it matters: many top AI infra roles quietly depend on excellent storage judgment. Google's current staff AI/ML role explicitly mentions globally distributed databases such as Spanner-like systems; Anthropic is hiring Staff+ roles specifically in databases; Databricks and Snowflake sit directly on large-scale data/storage systems. <sup>26</sup>

- **Study:** relational modeling; schema evolution; indexes; query planning; MVCC; isolation anomalies; WAL and crash recovery; change data capture; data modeling for event-driven systems; OLTP vs analytics paths; queues and logs; partitioning; hot-key mitigation.
- **Read and use:** PostgreSQL docs on **Indexes**, **Index Types**, **Multicolumn Indexes**, **MVCC**, **Transaction Isolation**, and **WAL**; Kafka introduction/design docs; DDIA Chapters 2–4 and 5–7. <sup>27</sup>
- **Practice set:** write and tune top-N-per-group SQL; pick indexes for 6 representative queries; reproduce a non-repeatable read and phantom-read case; explain PostgreSQL MVCC snapshots in your own words; compare B-tree vs GIN vs BRIN choices; design a CDC pipeline; design a multi-tenant schema for quotas and billing; interpret a slow query plan; choose between Kafka and database tables for a workflow; describe exactly how WAL improves crash safety.
- **Mini-projects:** create a PostgreSQL isolation-lab repo; write an indexing-and-query-plan case study; design a durable event-driven ingestion service with Kafka + Postgres.

## Networking — P1.

Why it matters: low-latency AI products and distributed control planes are networking problems in disguise. Databricks explicitly calls out HTTP, websockets, authentication, and browser/network fundamentals for some roles; gRPC, TLS, and HTTP version tradeoffs show up in real interviews and real postmortems. <sup>28</sup>

- **Study:** TCP vs application-layer semantics; HTTP/1.1 vs HTTP/2 vs HTTP/3; gRPC unary and streaming; load balancing; connection pooling; head-of-line blocking; client retries and retry budgets; request hedging; websocket and streaming tradeoffs; service discovery; DNS basics; CDN and cache invalidation.
- **Read and use:** IETF **RFC 9114** for HTTP/3; IETF **RFC 8446** for TLS 1.3; gRPC **Introduction** and **Core concepts** docs. <sup>15</sup>
- **Practice set:** trace a unary gRPC call; design a bidirectional streaming API; compare HTTP/2 vs HTTP/3 for a chat or agent product; analyze a TLS handshake capture; diagnose port exhaustion; tune keepalives and timeouts; design health checks and load balancer failover; explain backpressure in a streaming pipeline; design request fan-out with retry budgets; write a note on why CDN invalidation often dominates perceived latency.
- **Mini-projects:** build a small gRPC service with unary and streaming endpoints; capture and annotate a TLS 1.3 handshake; produce a benchmarking note comparing HTTP and gRPC paths for a toy service.

## Security — P1.

Why it matters: enterprise and public AI systems now live inside tight security, identity, and abuse-control boundaries. Anthropic has safeguards and identity/access roles; Snowflake highlights product security depth for distributed systems and AI/ML; Kubernetes, OAuth, JWT, and secure tool use show up in platform interviews and production requirements. <sup>29</sup>

- **Study:** authn vs authz; OAuth 2.0 flows; JWT structure and validation; secret management; least privilege; RBAC; network segmentation; abuse controls; supply-chain/security scanning; secure prompt/tool boundaries; threat modeling for model-serving systems.
- **Read and use:** **OAuth 2.0 RFC 6749**; **JWT RFC 7519**; **OWASP Top 10**; **OWASP ASVS**; Kubernetes **Security** docs. <sup>30</sup>
- **Practice set:** diagram auth code + PKCE; audit a JWT validation path; threat-model an inference API; review a Kubernetes RBAC policy; classify bugs using OWASP Top 10; design secrets rotation; write an abuse-mitigation plan for a public API; analyze a broken-access-control incident; design a secure MCP/tool sandbox; run a tabletop incident response exercise.
- **Mini-projects:** build a hardened OAuth/JWT demo service; publish a threat model for an LLM API with tool use; create a Kubernetes hardening checklist for AI workloads.

## ML systems, evals, and experimentation — P1.

Why it matters: frontier and high-paying AI roles increasingly differentiate on **measurement discipline**. OpenAI's developer docs emphasize evals, fine-tuning, and model optimization; Anthropic has extensive recent engineering writing on evals and agent quality; InstructGPT remains foundational for understanding post-training and alignment; experimentation is the bridge between "interesting model change" and "production value." <sup>31</sup>

- **Study:** offline vs online evaluation; task decomposition; regression testing; error taxonomies; retriever metrics; model graders; human eval loops; supervised fine-tuning; distillation; RLHF/DPO

concepts at a practical level; A/B test design, power, and guardrails; data quality and labeling strategy.

- **Read and use:** OpenAI **Working with evals**, **Model optimization**, **Supervised fine-tuning**, and distillation cookbook; Anthropic **Demystifying evals for AI agents** and **Building Effective Agents; InstructGPT; Understanding Experimentation Platforms** for experimentation operations. <sup>32</sup>
- **Practice set:** define an eval schema for tool use; design online/offline metrics for a reranker; build an error taxonomy from 100 samples; estimate minimum detectable effect for a product experiment; compare two prompts with rubric-based grading; run a tiny fine-tuning or distillation workflow; design a canary rollout for a model upgrade; write a data labeling guide; build a cost-quality Pareto chart; explain when you would choose preference modeling vs task-specific testing.
- **Mini-projects:** build an **eval dashboard** for a small LLM app; run a **distillation or SFT case study** with before/after metrics; write an **A/B experiment proposal** for an AI feature with primary and guardrail metrics.

### System design — P0.

Why it matters: at your target levels, many companies are trying to answer a deeper question than “can this person code?” They are asking whether you can reduce ambiguity, choose sensible tradeoffs, and design systems that scale operationally and organizationally. Microsoft explicitly evaluates design, and current staff AI/ML roles at Google and Databricks center on architecture and technical direction. <sup>12</sup>

- **Study:** requirement clarification; estimation; SLOs; capacity planning; APIs and contracts; storage choices; queues/logs; caching layers; observability; quotas/billing; failure domains; multitenancy; cost models; data retention; privacy and abuse surfaces.
- **Read and use:** DDIA Chapters 1–9 and 11; OpenTelemetry docs for traces/metrics/logs; Kafka docs; current OpenAI/Anthropic/Google technical posts on agents, evals, inference, and context. <sup>33</sup>
- **Practice set:** design an LLM serving platform; design a multi-tenant vector retrieval service; design usage metering and billing; design an experimentation platform; design telemetry ingestion and tracing; design a feature store with online serving; design a global rate limiter; design an agent orchestration platform with tools; design collaborative chat; design a notification service with retries and dedupe.
- **Mini-projects:** write three production-style design docs; present each aloud in 30 minutes and record yourself; create a “tradeoff appendix” pattern library showing storage, queue, cache, and consistency choices.

### Behavioral — P0.

Why it matters: Microsoft’s own rubric foregrounds collaboration, drive for results, customer focus, influence, judgment, and adaptability; Meta and Databricks both describe behavioral rounds as substantive parts of the process, not filler. For staff+ compensation, your story quality can be the difference between “high senior” and “staff-caliber.” <sup>13</sup>

- **Study:** ownership scope, conflict navigation, tradeoff communication, influence without authority, crisis handling, project selection, long-term technical bets, mentorship, and failure analysis.
- **Read and use:** Microsoft **Interview tips** and **Technical interviews** pages; Meta’s software-engineering interview prep materials; Databricks behavioral-interview guidance. <sup>34</sup>
- **Practice set:** answer 10 recurring prompts out loud — biggest failure, biggest system incident, hardest stakeholder conflict, most ambiguous project, disagree-and-commit, mentoring story, quality-improvement story, technical leadership story, customer-impact story, and “why this company now.”

- **Mini-projects:** build a written story bank; write a one-page impact inventory with metrics for each project; create a “promotion packet style” brag document that recruiters and interviewers can use as anchors.

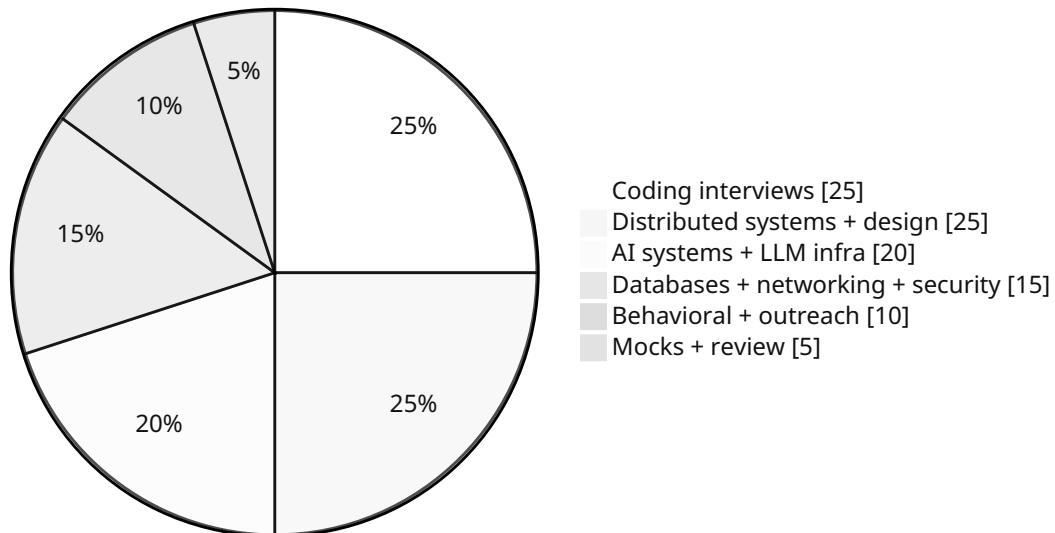
## Twelve-month operating cadence

The calendar below assumes a start in **July 2026** and is optimized for an experienced engineer targeting staff-caliber AI systems roles. The time split is a recommendation, but it is grounded in the public evidence above: current target roles and interview formats reward coding, large-scale systems, and AI-infra judgment far more than broad but shallow coverage. <sup>35</sup>

### Weekly allocation options

| Bucket                              | 5h/week | 10h/week | 15h/week |
|-------------------------------------|---------|----------|----------|
| Coding interviews                   | 1.25h   | 2.5h     | 4h       |
| Distributed systems + system design | 1.25h   | 2.5h     | 3.5h     |
| AI systems + LLM infra              | 1h      | 2h       | 3h       |
| Databases + networking + security   | 0.75h   | 1.5h     | 2.5h     |
| Behavioral + resume + outreach      | 0.5h    | 1h       | 1h       |
| Mocks + review                      | 0.25h   | 0.5h     | 1h       |

### Default 10-hour weekly allocation





## Representative daily rhythm

| Plan         | Mon           | Tue                         | Wed                           | Thu                             | Fri                            | Weekend                              |
|--------------|---------------|-----------------------------|-------------------------------|---------------------------------|--------------------------------|--------------------------------------|
| 5h/<br>week  | 60m<br>coding | 45m dist-<br>sys reading    | 60m AI infra                  | 45m DB/<br>network/<br>security | 30m<br>behavioral              | 60m mixed<br>review                  |
| 10h/<br>week | 90m<br>coding | 60m CS/<br>runtime or<br>DB | 90m<br>distributed<br>systems | 90m AI infra/<br>evals          | 60m<br>behavioral or<br>resume | 3h mock +<br>project + 90m<br>review |
| 15h/<br>week | 90m<br>coding | 90m dist-<br>sys            | 90m AI infra                  | 90m coding                      | 60m<br>behavioral              | 5h project +<br>mocks +<br>review    |

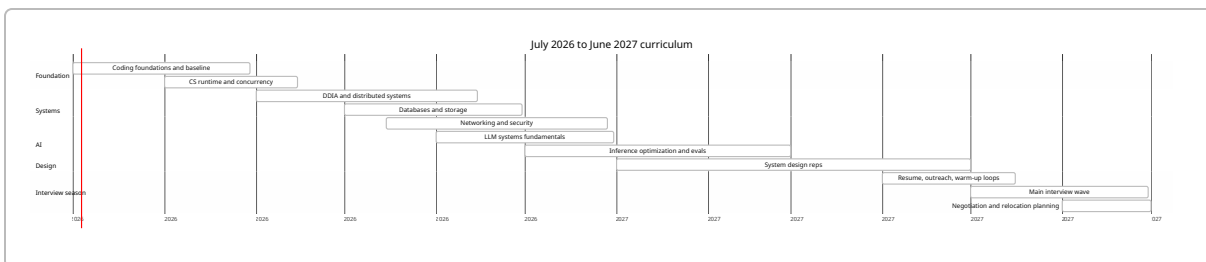
## Mock interview cadence

| Phase        | Coding<br>mocks | System-design<br>mocks | Behavioral<br>mocks | Goal                                                       |
|--------------|-----------------|------------------------|---------------------|------------------------------------------------------------|
| Months 1–3   | 1 / month       | 0–1 / month            | 1 / month           | Build baseline and remove rust                             |
| Months 4–6   | 2 / month       | 1 / month              | 1 / month           | Reach stable interview-readiness in coding and core design |
| Months 7–9   | 2 / month       | 2 / month              | 1 / month           | Shift from study to performance                            |
| Months 10–12 | Weekly          | Weekly                 | 2 / month           | Run like a real interview season                           |

## Monthly milestones and measurable outcomes

| Month          | Focus                                          | Measurable outcome                                                                     |
|----------------|------------------------------------------------|----------------------------------------------------------------------------------------|
| July 2026      | Baseline, coding foundations, story inventory  | Diagnostic coding set done; 10 coding problems solved cleanly; 10 story seeds captured |
| August 2026    | Trees, graphs, runtime/concurrency, OSTEP core | 20 additional coding problems; first concurrency mini-lab completed                    |
| September 2026 | DDIA foundations, Raft, first design reps      | 3 design docs drafted; 1 coding mock; 1 distributed-systems writeup                    |
| October 2026   | Databases, MVCC, WAL, Kafka, SQL tuning        | Postgres lab repo completed; 35–40 total coding problems; 1 design mock                |

| Month         | Focus                                                                             | Measurable outcome                                                                                              |
|---------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| November 2026 | LLM systems core: transformers, retrieval, vLLM, KV cache                         | First AI-systems mini-project live on GitHub; 1 AI-infra presentation to yourself/peer                          |
| December 2026 | Inference optimization: FlashAttention, speculative decoding, quantization, evals | Benchmark report comparing two inference optimizations; 2 mocks this month                                      |
| January 2027  | System design block I: API, cache, queues, observability                          | 6 total design prompts completed; OpenTelemetry or tracing exercise finished                                    |
| February 2027 | System design block II: RAG, agents, quotas, billing, multitenancy                | 10 total design prompts; second portfolio artifact finished                                                     |
| March 2027    | Eval, experimentation, distillation, rollout discipline                           | Eval dashboard or experiment proposal completed; 60–80 total coding problems                                    |
| April 2027    | Resume rewrite, networking, mock-heavy warm-up wave                               | Resume version for AI infra complete; 3 recruiter-ready portfolio links; 4 mocks completed                      |
| May 2027      | Main interview wave, frontier-lab/ company-specific prep                          | Weekly coding + design mocks sustained; 20 stories fully fleshed in STAR form                                   |
| June 2027     | Offer phase, gap-filling, negotiation, immigration logistics                      | 30 stories prepared; 120–160 total coding problems depending on weekly hours; 20+ total mocks if on 10–15h plan |



## Behavioral portfolio

Use the same **STAR backbone** for every story:

- **Situation:** business context, scale, stakeholders, constraint.
- **Task:** what you specifically owned or were accountable for.
- **Action:** the decisions you made, alternatives you considered, tradeoffs, and how you influenced others.
- **Result:** metrics, system outcomes, user impact, organizational effect, and what you learned.

This section exists because your target companies do not only ask whether you can solve problems; they also ask how you scope them, lead them, recover from failure, and raise the performance of teams around

you. Microsoft's published competencies are especially useful anchors: collaboration, results, customer focus, influence, judgment, and adaptability. <sup>36</sup>

## The 30 stories to prepare

| Story | Prompt                                                    | What to prepare                                                |
|-------|-----------------------------------------------------------|----------------------------------------------------------------|
| 1     | Most important production system you owned                | Scale, SLA/SLO, your design choices, measurable result         |
| 2     | A project where you reduced latency                       | Baseline, bottleneck, intervention, final numbers              |
| 3     | A project where you lowered cost                          | Cost driver, options considered, savings, tradeoffs            |
| 4     | A project where you improved quality/evals                | Error taxonomy, instrumentation, change in quality metric      |
| 5     | A telemetry or observability improvement                  | Before/after incident detectability, debugging speed, adoption |
| 6     | A cross-team initiative you led                           | Stakeholders, alignment mechanism, conflicts, final outcome    |
| 7     | A project with ambiguous requirements                     | How you framed the problem, de-risked it, and chose scope      |
| 8     | A design disagreement with another senior engineer        | Stakes, disagreement, how you argued, how you resolved it      |
| 9     | A failure that was your fault                             | What you missed, how you corrected course, durable lesson      |
| 10    | A severe production incident                              | Timeline, decision points, communication, postmortem actions   |
| 11    | A rollback or launch you stopped                          | Why you said no, evidence used, downstream impact              |
| 12    | A time you influenced without authority                   | How you got buy-in when you were not the formal owner          |
| 13    | A project that improved developer productivity            | Time saved, adoption, indirect leverage                        |
| 14    | A mentoring story                                         | Who you helped, what changed, evidence of growth               |
| 15    | A time you handled underperformance or poor collaboration | Sensitivity, fairness, concrete action, outcome                |
| 16    | A project you deliberately simplified                     | Why simplification beat sophistication, what you cut           |

| Story | Prompt                                                       | What to prepare                                                                    |
|-------|--------------------------------------------------------------|------------------------------------------------------------------------------------|
| 17    | A technically hard bug you solved                            | Root cause process, dead ends, instrumentation, lesson                             |
| 18    | A build-vs-buy decision                                      | Options, cost/risk analysis, long-term consequences                                |
| 19    | A project with data or model-quality problems                | Diagnosis, evaluation strategy, corrective actions                                 |
| 20    | A time you traded short-term speed for long-term reliability | Tension, communication, final payoff                                               |
| 21    | A time you traded long-term elegance for immediate delivery  | Why, how you mitigated debt, later cleanup                                         |
| 22    | A security, privacy, or compliance-sensitive project         | Risk surface, controls, technical and organizational judgment                      |
| 23    | A customer-impact story                                      | Who benefited, how you measured value, what changed                                |
| 24    | A project that required learning a new domain quickly        | Learning plan, execution, evidence you became effective                            |
| 25    | A time you made a wrong technical bet                        | Bad assumption, signal you noticed, correction path                                |
| 26    | A project that required roadmap or prioritization judgment   | What you said yes/no to, why, and resulting leverage                               |
| 27    | A project where you created reusable infrastructure          | Reuse story, adoption, maintenance implications                                    |
| 28    | A time you represented engineering to leadership             | How you translated complexity into decisions and tradeoffs                         |
| 29    | Why you want frontier AI or top AI-platform work now         | Motivation, fit, what you want to build, why now                                   |
| 30    | Your “career thesis” story                                   | The arc from Microsoft Applied Scientist to staff-level AI infrastructure engineer |

For each story, prepare a **90-second version**, a **3-minute version**, and a **deeper technical appendix**. The 90-second version gets you through screens; the 3-minute version survives follow-ups; the appendix gives you enough depth for senior interviewers.

## Company targeting, negotiation, and relocation

### Company comparison

The table below synthesizes current role pages and recent compensation signals. Compensation numbers are directional and use recent U.S. Levels.fyi data; titles and focus areas come from current public company job pages and interview-prep materials. <sup>37</sup>

| Company    | Strong-fit role titles                                                                                                    | Approx recent U.S. TC signal                                                      | Public interview or role signal                                                                                                                                                                                |
|------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OpenAI     | <b>Member of Technical Staff</b> , Senior Staff Software Engineer, Data Infrastructure, Real-Time, Compute Infrastructure | SWE median about <b>\$800k</b> ; upper levels above <b>\$1.2M</b> . <sup>38</sup> | Official pages emphasize large-scale data infrastructure, real-time systems, and compute clusters; OpenAI notes that research and engineering hires share the internal title <b>MTS</b> . <sup>39</sup>        |
| Anthropic  | Staff+ Software Engineer, Cloud Inference, Inference Runtime, Caching, Databases, AI Reliability, Research Tools          | Senior around <b>\$563k</b> ; lead around <b>\$841k</b> . <sup>40</sup>           | Technical roles use <b>live coding</b> in Colab/CodeSignal, and current openings concentrate heavily in inference/platform/reliability/database domains. <sup>41</sup>                                         |
| Meta       | Software Engineer, Machine Learning; AI SysML; Systems ML; technical leadership variants                                  | E6 about <b>\$703k</b> ; E7 about <b>\$1.24M</b> . <sup>42</sup>                  | Public prep materials point to 45-minute technical screens and multi-round onsite/full-loop interviews; current roles emphasize AI infrastructure and systems ML. <sup>43</sup>                                |
| Google     | Staff/Senior Staff Software Engineer, AI/ML; Staff SWE ML infrastructure                                                  | L7 about <b>\$829k</b> ; L8 about <b>\$1.12M</b> . <sup>44</sup>                  | Current staff AI/ML roles explicitly ask for distributed systems, ML lifecycle ownership, RAG, and inference cost/latency optimization. <sup>45</sup>                                                          |
| Databricks | Staff Software Engineer, Distributed Data Systems; Backend; AI Platform; Search Quality                                   | L6 staff about <b>\$1.05M</b> ; L7 can exceed that materially. <sup>46</sup>      | Databricks says interviews mix algorithms and CS fundamentals with hands-on coding, debugging, and role-specific systems/web knowledge; current staff roles emphasize large distributed systems. <sup>47</sup> |
| Snowflake  | Staff Software Engineer, Cortex AI Infrastructure; Feature Store; Control Plane                                           | IC5 up to roughly <b>\$1.18M</b> . <sup>48</sup>                                  | Current AI/ML engineering roles emphasize distributed systems, high-throughput APIs, real-time ML systems, and Cortex infrastructure. <sup>49</sup>                                                            |

| Company | Strong-fit role titles                                                    | Approx recent U.S. TC signal                                    | Public interview or role signal                                                                                                          |
|---------|---------------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Stripe  | Staff Software Engineer, Machine Learning Platform; ML Infrastructure     | L4 about <b>\$760k</b> ; L5 about <b>\$929k</b> . <sup>50</sup> | Current staff and ML-platform roles emphasize business impact under latency, reliability, cost, and security constraints. <sup>51</sup>  |
| NVIDIA  | Principal AI/ML Infra Software Engineer, GPU Clusters; DGX Cloud AI Infra | IC7 about <b>\$1.07M</b> . <sup>52</sup>                        | Current roles emphasize GPU clusters, cloud-native stack, runtime systems, performance, and large-scale AI infrastructure. <sup>53</sup> |

## Recommended application timing

For a 12-month plan, the cleanest sequencing is:

| Window       | What to do                                                                                  | Why                                                                                    |
|--------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Months 1–6   | Build core skill base and one visible portfolio artifact                                    | Too early to apply wastes recruiter bandwidth                                          |
| Months 7–8   | Rewrite resume, story bank, and GitHub/project READMEs; start warm outreach                 | This is when your evidence becomes recruiter-usable                                    |
| Months 9–10  | Run a warm-up wave at strong but non-peak targets                                           | Use these to expose gaps before expensive frontier-lab loops                           |
| Months 10–12 | Run the main wave at OpenAI, Anthropic, Meta, Google, Databricks, Snowflake, Stripe, NVIDIA | By then you should have enough reps, stories, and artifacts to compete at higher bands |

My ordering for interview risk is:

- **Warm-up wave:** Databricks, Snowflake, Stripe, NVIDIA, selected Google L6/L7 roles.
- **Peak wave:** Anthropic, OpenAI, Meta E6/E7, top Google Staff AI/ML roles.
- **Role-specific stretch wave:** the most specialized inference, runtime, or public-sector/secure-AI positions once your system-design answers are sharp.

That order is deliberately pragmatic. Databricks, Snowflake, Stripe, and NVIDIA are still extremely strong companies, but their public materials make it easier to infer role fit and prep. Anthropic and OpenAI should be treated as later-wave targets because the compensation is excellent and the bar is correspondingly high.

<sup>54</sup>

## Negotiation approach

Use negotiation as part of the plan, not an afterthought.

- **Negotiate total compensation, not just base salary.** Harvard and Levels both emphasize benchmarking the full package, and tech offers frequently move on equity, sign-on, level, and refreshers rather than base alone. <sup>55</sup>
- **Use competing offers honestly.** Levels notes that some companies may ask to see written proof of a competing offer, which is one reason not to bluff. <sup>56</sup>
- **Ask for time to review the full package.** This is standard negotiation hygiene and improves decision quality. <sup>57</sup>
- **Separate leveling from package optimization.** If you believe your scope maps to staff rather than senior, push for level calibration first; a level change often matters more than small package movement.
- **Do not assume negotiation kills the offer.** Research summarized by Harvard Business Review indicates that job seekers generally overestimate the risk that negotiating will jeopardize the offer. <sup>58</sup>
- **For private-company equity, ask harder questions.** Ask about refresh cadence, vesting behavior, secondary liquidity, strike price or 409A implications where relevant, and expected review cycles. Private-company headline TC is more uncertainty-laden than public-company TC. This is an inference from the way private-company packages are reported and valued. <sup>59</sup>

## Relocation and visa considerations

Because you may relocate to the United States, immigration timing should be part of the application strategy from the beginning, not after offer stage.

| Route              | Official signal                                                                                                                                                                                                                              | Practical implication for your plan                                                                                                                                                                                                 |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>L-1A / L-1B</b> | USCIS says L-1 is for intracompany transfers, and L-1B covers specialized-knowledge employees; L classification requires qualifying international-organization relationships and prior qualifying employment abroad. <sup>60</sup>           | Best if moving <b>within</b> a multinational like your current employer. It is <b>not</b> the default route for jumping to an unrelated new company.                                                                                |
| <b>H-1B</b>        | USCIS describes H-1B as the specialty-occupation category; the latest official cap-registration cycle for FY 2027 ran <b>March 4–19, 2026</b> , and USCIS says portability exists for eligible workers already in H-1B status. <sup>61</sup> | If you are <b>not already cap-counted</b> , cap-subject hiring is calendar-bound. Ask recruiters early about sponsorship strategy. If you <b>are already in H-1B status</b> , job mobility is easier because portability may apply. |

| Route       | Official signal                                                                                                                           | Practical implication for your plan                                                                                                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>O-1A</b> | USCIS describes O-1 as the route for individuals with extraordinary ability in sciences, education, business, or athletics. <sup>62</sup> | This is often the best premium-market alternative for senior technical talent with a strong evidence stack: publications, patents, public talks, open source, awards, or significant press/impact evidence. Build that dossier during the year. |
| <b>TN</b>   | USCIS says TN is for qualified <b>Canadian or Mexican citizens</b> . <sup>63</sup>                                                        | Permanent residence alone is <b>not</b> enough. If you are not a Canadian or Mexican citizen, TN is not currently your route.                                                                                                                   |

The practical advice is simple:

- If an **intracompany U.S. transfer** remains realistic, keep that path warm because L-1 can be the fastest route. <sup>64</sup>
- If you intend to move to a **new employer**, ask from the first recruiter conversation whether the company supports your likely route and on what timeline.
- Build an **O-1-ready evidence file** throughout the year anyway. Even if you never use it, the same artifacts help with recruiting: talks, papers, design docs, open source, patents, benchmarks, and visible leadership.

The highest-return version of this plan is therefore not “study everything.” It is:

1. **Become visibly excellent at coding, distributed systems, and LLM infrastructure.**
2. **Turn that excellence into three public or semi-public artifacts.**
3. **Package your Microsoft work into a staff-level narrative.**
4. **Run a staged interview season with negotiation and immigration logistics already planned.**

If you want one sentence to keep in mind for the full year, it is this: **prepare like a Staff AI infrastructure engineer, not like a generic interview candidate.**

<sup>1</sup> <sup>6</sup> <sup>37</sup> <sup>38</sup> OpenAI Software Engineer Salary | \$254K-\$1.23M+

[https://www.levels.fyi/companies/openai/salaries/software-engineer?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/openai/salaries/software-engineer?utm_source=chatgpt.com)

<sup>2</sup> <sup>7</sup> <sup>9</sup> <sup>11</sup> <sup>24</sup> <https://openai.com/careers/software-engineer-data-infrastructure-san-francisco/>

<https://openai.com/careers/software-engineer-data-infrastructure-san-francisco/>

<sup>3</sup> <sup>8</sup> <sup>35</sup> <sup>41</sup> <https://www.anthropic.com/careers>

<https://www.anthropic.com/careers>

<sup>4</sup> <sup>10</sup> <sup>14</sup> <sup>26</sup> <sup>45</sup> <https://www.google.com/about/careers/applications/jobs/results/115305015559496390-staff-software-engineer-machine-learning-google-chat>

<https://www.google.com/about/careers/applications/jobs/results/115305015559496390-staff-software-engineer-machine-learning-google-chat>

<sup>5</sup> <sup>39</sup> <https://openai.com/careers/senior-staff-software-engineer-gov-washington-dc/>

<https://openai.com/careers/senior-staff-software-engineer-gov-washington-dc/>



- 12 <https://careers.microsoft.com/v2/global/en/hiring-tips/technical-interviewing>  
<https://careers.microsoft.com/v2/global/en/hiring-tips/technical-interviewing>
- 13 34 36 <https://careers.microsoft.com/v2/global/en/hiring-tips/interview-tips.html>  
<https://careers.microsoft.com/v2/global/en/hiring-tips/interview-tips.html>
- 15 <https://datatracker.ietf.org/doc/html/rfc8446>  
<https://datatracker.ietf.org/doc/html/rfc8446>
- 16 <https://owasp.org/www-project-top-ten/>  
<https://owasp.org/www-project-top-ten/>
- 17 31 32 <https://developers.openai.com/api/docs/guides/evals>  
<https://developers.openai.com/api/docs/guides/evals>
- 18 22 <https://pages.cs.wisc.edu/~remzi/OSTEP/>  
<https://pages.cs.wisc.edu/~remzi/OSTEP/>
- 19 20 <https://leetcode.com/studyplan/top-interview-150/>  
<https://leetcode.com/studyplan/top-interview-150/>
- 21 28 47 54 <https://www.databricks.com/blog/2020/01/22/engineering-interviews-a-hiring-managers-guide.html>  
<https://www.databricks.com/blog/2020/01/22/engineering-interviews-a-hiring-managers-guide.html>
- 23 33 <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>  
<https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>
- 25 **Attention Is All You Need**  
[https://arxiv.org/abs/1706.03762?utm\\_source=chatgpt.com](https://arxiv.org/abs/1706.03762?utm_source=chatgpt.com)
- 27 <https://www.postgresql.org/docs/current/indexes.html>  
<https://www.postgresql.org/docs/current/indexes.html>
- 29 <https://www.anthropic.com/jobs?ref=producthunt>  
<https://www.anthropic.com/jobs?ref=producthunt>
- 30 <https://datatracker.ietf.org/doc/html/rfc6749>  
<https://datatracker.ietf.org/doc/html/rfc6749>
- 40 **Anthropic Software Engineer Salary | \$563K-\$841K+**  
[https://www.levels.fyi/companies/anthropic/salaries/software-engineer?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/anthropic/salaries/software-engineer?utm_source=chatgpt.com)
- 42 **Meta Software Engineer Salary | \$178K-\$4.36M+**  
[https://www.levels.fyi/companies/meta/salaries/software-engineer?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/meta/salaries/software-engineer?utm_source=chatgpt.com)
- 43 **Preparing for Your Technical Screen**  
[https://www.metacareers.com/swe-prep-techscreen/?utm\\_source=chatgpt.com](https://www.metacareers.com/swe-prep-techscreen/?utm_source=chatgpt.com)
- 44 **Google Salaries**  
[https://www.levels.fyi/companies/google/salaries?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/google/salaries?utm_source=chatgpt.com)
- 46 **Databricks Software Engineer Salary | \$250K-\$1.83M+**  
[https://www.levels.fyi/companies/databricks/salaries/software-engineer?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/databricks/salaries/software-engineer?utm_source=chatgpt.com)
- 48 **Snowflake Software Engineer Salary | \$238K-\$1.18M+**  
[https://www.levels.fyi/companies/snowflake/salaries/software-engineer?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/snowflake/salaries/software-engineer?utm_source=chatgpt.com)

49 <https://careers.snowflake.com/us/en/job/SNCOUS37DCF7EA43424BB899A6C99065A9897AEXTERNALENUSA1F1779B464C48F6A66456314825C8AC/Staff-Software-Engineer-Cortex-AI-Infrastructure>

[https://careers.snowflake.com/us/en/job/](https://careers.snowflake.com/us/en/job/SNCOUS37DCF7EA43424BB899A6C99065A9897AEXTERNALENUSA1F1779B464C48F6A66456314825C8AC/Staff-Software-Engineer-Cortex-AI-Infrastructure)

[SNCOUS37DCF7EA43424BB899A6C99065A9897AEXTERNALENUSA1F1779B464C48F6A66456314825C8AC/Staff-Software-Engineer-Cortex-AI-Infrastructure](https://careers.snowflake.com/us/en/job/SNCOUS37DCF7EA43424BB899A6C99065A9897AEXTERNALENUSA1F1779B464C48F6A66456314825C8AC/Staff-Software-Engineer-Cortex-AI-Infrastructure)

#### 50 Stripe Salaries

[https://www.levels.fyi/companies/stripe/salaries?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/stripe/salaries?utm_source=chatgpt.com)

51 <https://stripe.com/jobs/listing/staff-software-engineer-machine-learning-platform/7712852>

<https://stripe.com/jobs/listing/staff-software-engineer-machine-learning-platform/7712852>

#### 52 Nvidia Salaries

[https://www.levels.fyi/companies/nvidia/salaries?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/nvidia/salaries?utm_source=chatgpt.com)

53 <https://jobs.nvidia.com/careers/job/893394928067>

<https://jobs.nvidia.com/careers/job/893394928067>

55 <https://online.hbs.edu/blog/post/salary-negotiation-tips>

<https://online.hbs.edu/blog/post/salary-negotiation-tips>

56 <https://www.levels.fyi/blog/ultimate-negotiation-guide.html>

<https://www.levels.fyi/blog/ultimate-negotiation-guide.html>

57 [https://www.linkedin.com/posts/harvard-business-review\\_15-rules-for-negotiating-a-job-offer-activity-7387935994160762880-v9up](https://www.linkedin.com/posts/harvard-business-review_15-rules-for-negotiating-a-job-offer-activity-7387935994160762880-v9up)

[https://www.linkedin.com/posts/harvard-business-review\\_15-rules-for-negotiating-a-job-offer-activity-7387935994160762880-v9up](https://www.linkedin.com/posts/harvard-business-review_15-rules-for-negotiating-a-job-offer-activity-7387935994160762880-v9up)

58 <https://hbr.org/2024/05/research-negotiating-is-unlikely-to-jeopardize-your-job-offer>

<https://hbr.org/2024/05/research-negotiating-is-unlikely-to-jeopardize-your-job-offer>

#### 59 Anthropic Salaries

[https://www.levels.fyi/companies/anthropic/salaries?utm\\_source=chatgpt.com](https://www.levels.fyi/companies/anthropic/salaries?utm_source=chatgpt.com)

60 64 <https://www.uscis.gov/policy-manual/volume-2-part-i-chapter-1>

<https://www.uscis.gov/policy-manual/volume-2-part-i-chapter-1>

61 <https://www.uscis.gov/working-in-the-united-states/h-1b-specialty-occupations>

<https://www.uscis.gov/working-in-the-united-states/h-1b-specialty-occupations>

62 <https://www.uscis.gov/working-in-the-united-states/temporary-workers/o-1-visa-individuals-with-extraordinary-ability-or-achievement>

<https://www.uscis.gov/working-in-the-united-states/temporary-workers/o-1-visa-individuals-with-extraordinary-ability-or-achievement>

63 <https://www.uscis.gov/working-in-the-united-states/temporary-workers/tn-usmca-professionals>

<https://www.uscis.gov/working-in-the-united-states/temporary-workers/tn-usmca-professionals>